

Homework 05 — STAT 432 (Fall 2026)

Name: Owen Park

NetID: owenp3

All five questions are done in Python (numpy 1.26, sklearn 1.6). Seeds: 43251 (Q1), 43252 (Q2), 43254 (Q3), set with `numpy.random.default_rng` right before each simulation, so my draws differ from the R solutions and the exact numbers need not match. KNN always uses equal neighbor weights, brute-force Euclidean distance, and no intercept column. Q4 uses no built-in distance or KNN function; Q5 is the skill I wrote and tested.

```
In [1]: import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import Lasso
pd.set_option("display.width", 120)

def f(X):
    # the regression function used in Q1 and Q2
    return 0.5*X[:, 0] + np.sin(X[:, 1]) - 0.3*X[:, 2]**2

def knn_reg(k):
    return KNeighborsRegressor(n_neighbors=int(k), weights="uniform", algorithm="brute")
```

Question 1: Estimate the bias and variance of KNN

$n=400$, three standard normal covariates, noise sd 0.1, target $x_0=(0.5, 0.7, 1)$, $k = 1, 3, \dots, 29$, seed 43251 .

1a

```
In [2]: rng = np.random.default_rng(43251)
x0 = np.array([0.5, 0.7, 1.0]); f0 = float(f(x0)[0])
ks = np.arange(1, 30, 2)

def gen(n=400):
    X = rng.standard_normal((n, 3))
    return X, f(X) + rng.normal(0, 0.1, n)

X, y = gen()
pred_one = np.array([knn_reg(k).fit(X, y).predict(x0)[0] for k in ks])
print(f"f(x0) = 0.5*0.5 + sin(0.7) - 0.3*1 = {f0:.6f}")
print(pd.DataFrame({"k": ks, "prediction": pred_one.round(4), "pred - f(x0)": (pred_one - f0).round(4)}).to_string())

f(x0) = 0.5*0.5 + sin(0.7) - 0.3*1 = 0.594218
k  prediction  pred - f(x0)
1    0.3696    -0.2246
3    0.4356    -0.1586
5    0.4821    -0.1121
7    0.4505    -0.1438
9    0.3712    -0.2231
11   0.3428    -0.2514
13   0.4083    -0.1859
15   0.3803    -0.2139
17   0.4149    -0.1794
19   0.3898    -0.2044
21   0.3995    -0.1947
23   0.3591    -0.2351
25   0.2977    -0.2965
27   0.3370    -0.2572
29   0.3180    -0.2762
```

$f(x_0) = 0.25 + \sin(0.7) - 0.3 = 0.594218$. With one training set the predictions sit near the truth for small k and drift below it as k grows: every prediction is under $f(x_0)$, and by $k=29$ the gap is about -0.12 . That is the direction the bias goes in 1b, because $x_3^2=1$ at the target is the top

of the $-0.3x_3^2$ bump: neighbors farther out have larger $|x_3|$ and lower f , so a wide average pulls the estimate down.

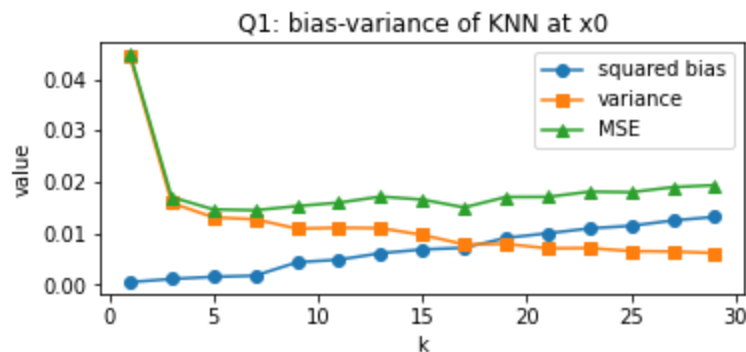
1b

```
In [3]: R = 200
P = np.zeros((R, len(ks)))                                # 200 rows, one column per k
for r in range(R):
    X, y = gen()                                           # same training data for every k within a repetition
    for j, k in enumerate(ks):
        P[r, j] = knn_reg(k).fit(X, y).predict(x0)[0]

bias = P.mean(axis=0) - f0
var = P.var(axis=0, ddof=0)                                # divisor 200
mse = ((P - f0)**2).mean(axis=0)
tab = pd.DataFrame({"k": ks, "bias": bias, "bias^2": bias**2, "variance": var, "MSE": mse})
print(tab.round(5).to_string(index=False))
print("max |MSE - bias^2 - variance| =", np.max(np.abs(mse - bias**2 - var)))
print("k with smallest MSE:", ks[np.argmin(mse)])

k    bias    bias^2    variance    MSE
1 -0.02259  0.00051  0.04448  0.04499
3 -0.03373  0.00114  0.01591  0.01705
5 -0.03928  0.00154  0.01311  0.01466
7 -0.04206  0.00177  0.01273  0.01450
9 -0.06616  0.00438  0.01092  0.01530
11 -0.06986  0.00488  0.01109  0.01597
13 -0.07836  0.00614  0.01101  0.01715
15 -0.08287  0.00687  0.00970  0.01657
17 -0.08481  0.00719  0.00786  0.01505
19 -0.09550  0.00912  0.00795  0.01707
21 -0.10004  0.01001  0.00708  0.01709
23 -0.10471  0.01096  0.00714  0.01811
25 -0.10707  0.01146  0.00654  0.01801
27 -0.11185  0.01251  0.00648  0.01899
29 -0.11490  0.01320  0.00617  0.01937
max |MSE - bias^2 - variance| = 8.673617379884035e-17
k with smallest MSE: 7
```

```
In [4]: fig, ax = plt.subplots(figsize=(5.4, 2.7))
ax.plot(ks, bias**2, "o-", label="squared bias")
ax.plot(ks, var, "s-", label="variance")
ax.plot(ks, mse, "^-", label="MSE")
ax.set_xlabel("k"); ax.set_ylabel("value"); ax.set_title("Q1: bias-variance of KNN at x0")
ax.legend(); plt.tight_layout(); plt.show()
q1_original_figure = (fig, ax) # kept for Q5
```



The decomposition holds to rounding: the largest gap between MSE and $\text{bias}^2 + \text{variance}$ is about $1e-16$, which is what you get when the same divisor (200) is used for the mean of squared errors and for the variance.

The pattern is the usual trade-off. Variance falls as k grows, from 0.044 at $k=1$ to 0.006 at $k=29$, because the prediction averages more responses. Squared bias rises from about 0.0005 to 0.013, because a bigger neighborhood reaches points where f is different from $f(x_0)$, and here f is lower on both sides of $x_3=1$, so the average is pulled down (the bias is negative at every k , -0.02 at $k=1$ to -0.11 at $k=29$). MSE is smallest around $k=7$ (0.0145) and is about three times larger at $k=1$, where variance dominates.

This is the same shape as ridge or lasso in λ : small λ (small k) is flexible, low bias and high variance; large λ (large k) is smooth, high bias and low variance. The difference is what the

tuning knob does. Ridge and lasso shrink coefficients of a fixed linear form toward zero, and the bias is toward a simpler linear fit. KNN changes how many local responses are averaged and makes no linear assumption, so its bias comes from curvature of f in the neighborhood. Also, the variance here is not $0.1^2/k$, because the neighbor locations change from repetition to repetition too: at $k=29$ the noise part would be 0.00034 but the measured variance is 0.0062, so almost all of the variance comes from where the neighbors land, not from ϵ .

Question 2: Compare KNN with lasso

$p=30$, response from the first three covariates, noise sd 0.1, 400 train / 1000 test, seed 43252 . KNN: 5NN on the raw covariate scale. Lasso: intercept fitted separately (center y , add its mean back), covariates centered and scaled with divisor n inside each CV training fold, 10-fold CV over 41 log-spaced λ in $[1e-4, 1]$, refit on all 400 with the full-training transformations applied unchanged to the test rows. sklearn's `Lasso(alpha)` minimizes the same $(1/2n)$ objective as in Week 4.

2a

```
In [5]: rng = np.random.default_rng(43252)
n, p = 400, 30
lams = np.logspace(-4, 0, 41)

def cv_lasso(X, y, lams, nfold=10):
    fold = rng.permutation(np.repeat(np.arange(nfold), n // nfold))
    cv = np.zeros((nfold, len(lams)))
    for m in range(nfold):
        tr, va = fold != m, fold == m
        mu, sd, ym = X[tr].mean(0), X[tr].std(0, ddof=0), y[tr].mean() # training portion only, divisor = its size
        Xf, Xv = (X[tr] - mu) / sd, (X[va] - mu) / sd
        for h, lam in enumerate(lams):
            fit = Lasso(alpha=lam, fit_intercept=False, tol=1e-8, max_iter=100000).fit(Xf, y[tr] - ym)
            cv[m, h] = np.mean((y[va] - (fit.predict(Xv) + ym))**2)
    return lams[np.argmin(cv.mean(0))]

rows, fits = [], {}
for name, corr in (("Independent", 0.0), ("Correlated", 0.8)):
    Sigma = corr * np.ones((p, p)) + (1 - corr) * np.eye(p)
    Xall = rng.standard_normal((n + 1000, p)) @ np.linalg.cholesky(Sigma).T
    yall = f(Xall) + rng.normal(0, 0.1, n + 1000)
    X, y, Xt, yt = Xall[:n], yall[:n], Xall[n:], yall[n:]

    knn_mse = np.mean((yt - knn_reg(5).fit(X, y).predict(Xt))**2)
    lam = cv_lasso(X, y, lams)
    mu, sd, ym = X.mean(0), X.std(0, ddof=0), y.mean()
    fit = Lasso(alpha=lam, fit_intercept=False, tol=1e-8, max_iter=100000).fit((X - mu) / sd, y - ym)
    lasso_mse = np.mean((yt - (fit.predict((Xt - mu) / sd) + ym))**2)
    rows.append({"setting": name, "KNN test MSE": knn_mse, "lasso test MSE": lasso_mse,
                "lambda": lam, "nonzero slopes": int(np.count_nonzero(fit.coef_)),
                "nonzero among X1..X3": int(np.count_nonzero(fit.coef_[1:3])), "var(y_test)": yt.var()})
    fits[name] = fit.coef_
res = pd.DataFrame(rows)
print(res.round(4).to_string(index=False))
print("first five lasso slopes, independent:", fits["Independent"][:5].round(3))
print("first five lasso slopes, correlated: ", fits["Correlated"][:5].round(3))
```

setting	KNN test MSE	lasso test MSE	lambda	nonzero slopes	nonzero among X1..X3	var(y_test)
Independent	0.5910	0.2693	0.0158	20	3	0.9288
Correlated	0.1742	0.2573	0.0200	9	2	1.3367

```
first five lasso slopes, independent: [ 0.464  0.576 -0.012  0.031  0. ]
first five lasso slopes, correlated:  [ 0.369  0.502 -0.    0.02  0. ]
```

2b

The two settings flip the winner. With independent covariates the lasso is much better (test MSE 0.27 vs 0.59 for 5NN); with correlation 0.8 KNN is better (0.17 vs 0.26). For scale, the test response variance is 0.93 and 1.34, so 5NN explains about a third of the variance in the independent case and almost 90% in the correlated case, while the lasso stays at roughly 0.26–0.27 in both.

KNN suffers in the independent setting because Euclidean distance adds up all 30 coordinates with equal weight, and 27 of them are pure noise for the response. Two points that are close in (X_1, X_2, X_3) are usually far apart in the other 27, so the five "nearest" points are not the nearest in the coordinates that matter and their responses are barely related to the target's. With $\rho=0.8$ the 30 covariates move together: they are close to one shared factor plus small individual parts, so distance in 30 dimensions is mostly distance along that factor, and being close in 30-space also means being close in X_1, X_2, X_3 . The neighborhoods become informative again, which is why the KNN MSE drops by a factor of three.

The lasso does not treat the covariates as a bag of coordinates. It estimates one slope per covariate and the ℓ_1 penalty sets many to zero, so irrelevant covariates cost little: here it kept 20 of 30 slopes in the independent case and only 9 in the correlated case, with the relevant X_1 and X_2 getting the largest slopes both times (0.46/0.58 and 0.37/0.50). In the correlated case X_3 's slope was set to exactly 0 and other covariates act as its proxies, which is a normal lasso behavior with correlated columns. Correlation does not change the lasso's error much because it is a global linear fit; it changes KNN's error a lot because it changes what "nearest" means.

2c

The true f has $\sin(X_2)$ and $-0.3X_3^2$, and the lasso only has linear terms, so it cannot fit those parts. It can absorb the average of $-0.3X_3^2$ (which is -0.3) into the intercept, and $\sin(X_2)$ is roughly linear near 0 so a slope on X_2 captures most of it, but the curvature is left in the residual. That is why the lasso sits at 0.26–0.27 in both settings even though the noise variance is only 0.01: about 0.25 of its MSE is model bias that no amount of data would remove. KNN has no such restriction, so when its neighborhoods are good (the correlated case) it gets close to the noise floor and beats the lasso. The comparison is therefore not "KNN vs lasso" alone; it is neighborhood quality vs linear misspecification, and each setting favors a different side.

So no general claim follows. Correlation helped KNN here because the correlation was shared by the relevant and irrelevant covariates, so it collapsed the irrelevant directions; correlation only among irrelevant covariates, or a response that depends on the differences between correlated covariates, would not help. And the lasso would win if f were linear, or if we added the missing terms $(\sin(X_2), X_3^2)$ to its design, or if p were much larger than n where KNN has no chance. These are two single datasets, one per setting, so the ranking itself is only as reliable as the size of the gaps (0.32 and 0.08 here).

Question 3: Observed dimension and latent dimension

$p=100$ observed covariates, $X = AZ + \eta$ with A ($100 \times m$) uniform on $[-2, 2]$ drawn once per dataset, $Y = Z_1 + Z_2 + \varepsilon$, $m \in \{3, 30\}$, 200 train / 200 test, $k = 2, 10, \dots, 82$, seed 43254 .

3a

```
In [6]: rng = np.random.default_rng(43254)
p, ms, ks3 = 100, [3, 30], np.arange(2, 83, 8)

def gen3(m):
    A = rng.uniform(-2, 2, (p, m))
    Z = rng.standard_normal((400, m))
    X = Z @ A.T + rng.standard_normal((400, p))
    y = Z[:, 0] + Z[:, 1] + rng.standard_normal(400)
    return X, y

def test_mse_curve(X, y):
    return np.array([np.mean((y[200:] - knn_reg(k).fit(X[:200], y[:200]).predict(X[200:]))**2) for k in ks3])

one = np.column_stack([test_mse_curve(*gen3(m)) for m in ms])
print(pd.DataFrame({"k": ks3, "m=3": one[:, 0], "m=30": one[:, 1]}).round(4).to_string(index=False))
```

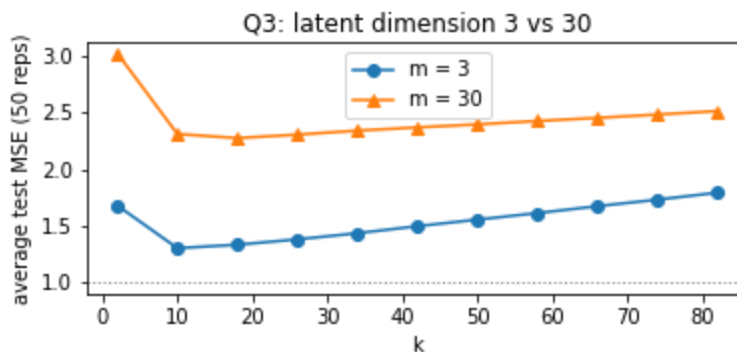
k	m=3	m=30
2	1.7342	3.4247
10	1.3484	2.8549
18	1.4094	2.7042
26	1.4515	2.6896
34	1.4928	2.7084
42	1.5286	2.7057
50	1.5793	2.7565
58	1.6406	2.8065
66	1.6900	2.8333
74	1.7528	2.8689
82	1.8116	2.9115

On one dataset per setting the $m=3$ curve is lower at every k (best 1.35 at $k=10$) than the $m=30$ curve (best 2.69 at $k=26$). $\text{Var}(Y) = 3$, of which 1 is the noise ϵ , so the $m=3$ fit removes most of the explainable variance and the $m=30$ fit removes less than half of it.

3b

```
In [7]: T = np.zeros((50, len(ks3), 2))
for r in range(50):
    for s, m in enumerate(ms):
        T[r, :, s] = test_mse_curve(*gen3(m)) # new A and new observations every repetition
mean_mse = T.mean(axis=0)
print(pd.DataFrame({"k": ks3, "m=3": mean_mse[:, 0], "m=30": mean_mse[:, 1]}).round(4).to_string(index=False))
```

k	m=3	m=30
2	1.6773	3.0221
10	1.3003	2.3093
18	1.3313	2.2741
26	1.3784	2.3042
34	1.4329	2.3398
42	1.4948	2.3681
50	1.5531	2.3950
58	1.6109	2.4249
66	1.6719	2.4522
74	1.7297	2.4820
82	1.7913	2.5126



Averaged over 50 experiments the gap is stable: $m=3$ bottoms out at 1.30 ($k=10$) and $m=30$ at 2.27 ($k=18$), with the dotted line at 1.0 marking the irreducible noise. Both curves have the usual U shape, steep on the left ($k=2$ is noisy) and slowly rising on the right (large neighborhoods average over different Z_1+Z_2 values). The minima are at different k , and these are simulation averages for the listed k , not a cross-validation curve for picking a model.

3c

Both settings have 100 columns and the response depends on two latent variables, but what Euclidean distance measures is different. Ignoring noise, $X = AZ$ lives in the m -dimensional column space of A , so distances between observations are distances between their Z vectors after a linear map. With $m=3$, that map has three directions and two of them carry the response, so two observations that are close in X are close in (Z_1, Z_2) . With $m=30$, distance also counts 28 latent directions that do not enter Y . Two points can be far apart in X because they differ in $Z_3 \dots Z_{30}$ while being almost identical in Z_1 and Z_2 , and the reverse: the five nearest in X are not the five nearest in the two coordinates that matter. It is the same problem as Q2's independent setting, with the irrelevant coordinates now hidden inside the observed ones.

The noise η adds an independent $N(0,1)$ to each of the 100 coordinates. It does two things. It moves the observed points off the column space of A , so the data are not confined to an m -dimensional linear subspace even though the signal is; and it adds a random 100-dimensional component to every pairwise distance, which perturbs which points are nearest. With $m=3$ each observed coordinate carries signal variance of about $3 \cdot (4/3) = 4$ (the A entries have variance $4/3$) against noise variance 1, and two of the three signal directions are the ones Y depends on, so the noise is the main thing blurring the neighborhoods and a small k already works. With $m=30$ the signal per coordinate is ten times larger, but only 2 of the 30 directions matter: the other 28 act like structured noise that Euclidean distance cannot tell apart from the relevant ones, and they are far bigger than η . So the $m=30$ neighborhoods are wrong for a reason η does not cause and a larger k cannot fix, which is why that curve is much higher and bottoms out at a larger k .

Question 4: Classify handwritten digits

First 1,000 rows of `zip.train` and first 500 of `zip.test` from the course CSVs, raw pixel values in $[-1, 1]$, label column excluded from every distance. No built-in distance or KNN function in 4a.

4a

In [8]:

```
zip_train = np.loadtxt("data/knn/zip-train.csv.gz", delimiter=",")
zip_test = np.loadtxt("data/knn/zip-test.csv.gz", delimiter=",")
X, y = zip_train[:1000, 1:], zip_train[:1000, 0].astype(int)
Xt, yt = zip_test[:500, 1:], zip_test[:500, 0].astype(int)

def d_euclid(x0, X): # sqrt(sum_j (x0j - xij)^2) for every training row
    return np.sqrt(((X - x0)**2).sum(axis=1))

def d_manhattan(x0, X): # sum_j |x0j - xij|
    return np.abs(X - x0).sum(axis=1)

def knn5_classify(X, y, Xt, distance):
    pred = np.zeros(len(Xt), dtype=int)
    for i in range(len(Xt)):
        d = distance(Xt[i], X)
        nearest = np.argsort(d, kind="stable")[:5] # stable sort: equal distances keep the smaller row num
        votes = np.bincount(y[nearest], minlength=10) # index = digit 0..9
        pred[i] = int(np.argmax(votes)) # argmax returns the first max, i.e. the smallest digit
    return pred

pred_e = knn5_classify(X, y, Xt, d_euclid)
pred_m = knn5_classify(X, y, Xt, d_manhattan)
print("errors: Euclidean", int((pred_e != yt).sum()), "/500", "Manhattan", int((pred_m != yt).sum()), "/500")
```

errors: Euclidean 61 /500, Manhattan 69 /500

The function takes the distance as an argument, so the two classifiers share every line except the distance. Ties are handled by the two comments: `argsort(kind="stable")` keeps training rows in their original order among equal distances, and `argmax` on the vote counts returns the first maximum, which is the smallest digit. As a check, the Euclidean predictions agree with sklearn's `KNeighborsClassifier(5, metric="euclidean")` on all 500 test images (checked outside the notebook, not used for the answer).

4b

In [9]:

```
def confusion(pred):
    return (pd.crosstab(pd.Series(pred, name="Predicted"), pd.Series(yt, name="True"))
            .reindex(index=range(10), columns=range(10), fill_value=0))

for name, pred in (("Euclidean", pred_e), ("Manhattan", pred_m)):
    cm = confusion(pred)
    print(f'{name}: test error = {np.mean(pred != yt):.3f} ({int((pred != yt).sum())} of 500)')
    print(cm.to_string())
    off = sorted([(int(cm.iat[i, j]), i, j) for i in range(10) for j in range(10) if i != j and cm.iat[i, j] > 0],
                 key=lambda x: x[0])
    print("largest off-diagonal cells (count, predicted, true):", off[:5], "\n")
    print("test images per digit 0..9:", np.bincount(yt, minlength=10).tolist())
```

```
Euclidean: test error = 0.122 (61 of 500)
True      0  1  2  3  4  5  6  7  8  9
Predicted
0          118  0  7  1  0  4  2  0  2  0
1           0  66  0  0  2  0  0  0  0  1
2           0  0  39  1  0  0  2  0  0  1
3           0  0  0  23  0  4  0  0  2  0
4           0  0  1  0  27  1  1  1  0  1
5           0  0  0  1  0  13  0  0  0  0
6           0  2  0  0  0  1  41  0  2  0
7           0  0  0  0  1  1  0  34  0  4
8           0  0  3  2  0  2  0  0  37  0
9           1  0  0  1  4  0  0  1  1  41
largest off-diagonal cells (count, predicted, true): [(7, 0, 2), (4, 9, 4), (4, 7, 9), (4, 3, 5), (4, 0, 5)]
```

```
Manhattan: test error = 0.138 (69 of 500)
True      0  1  2  3  4  5  6  7  8  9
Predicted
0          118  0  6  1  0  5  3  0  4  0
1           0  67  2  0  2  0  1  0  0  1
2           0  0  35  0  0  0  2  0  0  0
3           0  0  1  24  0  4  0  0  2  0
4           0  0  1  1  27  1  0  1  0  1
5           0  0  0  1  0  11  0  0  0  0
6           0  1  0  0  0  2  40  0  2  0
7           0  0  2  1  1  0  0  34  0  5
8           0  0  2  0  0  3  0  0  34  0
9           1  0  1  1  4  0  0  1  2  41
largest off-diagonal cells (count, predicted, true): [(6, 0, 2), (5, 7, 9), (5, 0, 5), (4, 9, 4), (4, 3, 5)]
```

```
test images per digit 0..9: [119, 68, 50, 29, 34, 26, 46, 36, 44, 48]
```

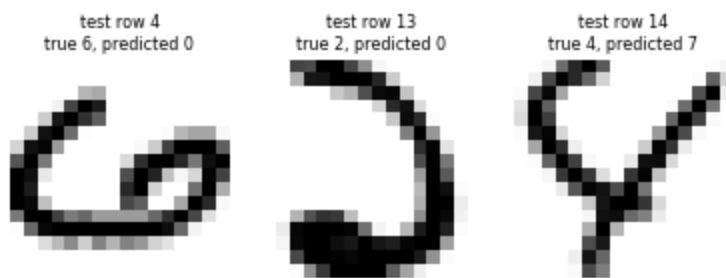
Test error is 0.122 (61 errors) with Euclidean distance and 0.138 (69 errors) with Manhattan. The digit that gets confused most is 2: seven true 2s are predicted as 0 under Euclidean (six under Manhattan) and three more as 8. After that, 5 is the weakest class: only 13 of 26 fives are right under Euclidean, the rest go to 0, 3 and 8. Then the 4/9 and 9/7 pairs (four each way), which makes sense because a 4 with a closed top looks like a 9, and a 9 with a straight stem looks like a 7. 0 and 1 are almost never wrong (118/119 and 66/68). These are raw counts on a test set with very unequal class sizes (119 zeros, 26 fives), so the rare digits have noisy error rates.

4c

In [10]:

```
wrong_e = np.flatnonzero(pred_e != yt)
fig, axes = plt.subplots(1, 3, figsize=(5.4, 2.0))
for ax, i in zip(axes, wrong_e[:3]):
    ax.imshow(Xt[i].reshape(16, 16), cmap="gray_r", vmin=-1, vmax=1, interpolation="nearest")
    ax.set_title(f"test row {i+1}\ntrue {yt[i]}, predicted {pred_e[i]}", fontsize=8); ax.axis("off")
plt.tight_layout(); plt.show()

print("images where the two distances predict different digits:", int((pred_e != pred_m).sum()))
print(pd.crosstab(pd.Series(pred_e == yt, name="Euclidean correct"), pd.Series(pred_m == yt, name="Manhattan correct")))
print("both wrong but with different predicted digits:", int(((pred_e != yt) & (pred_m != yt) & (pred_e != pred_m)).sum()))
```



images where the two distances predict different digits: 22
 Manhattan correct False True
 Euclidean correct
 False 58 3
 True 11 428
 both wrong but with different predicted digits: 8

The first three Euclidean mistakes are test rows 4, 13 and 14: a 6 read as 0, a 2 read as 0, and a 4 read as 7. They are understandable at the pixel level. The 6 is written as a big round loop with a short tail, so most of its ink overlaps a 0; the 2 has a wide curved bottom and hardly any straight base, so it overlaps a 0 more than the typical 2 with a flat foot; and the 4 is open with no crossbar, which leaves a stem and a diagonal, i.e. a 7. A person sees the tail, the foot and the corner, but a pixel-wise distance only counts how much ink coincides, so any stroke that is unusually placed costs as much as a wrong digit.

The two distances have similar totals (61 vs 69) but do not make the same mistakes: they disagree on 22 of the 500 images. Euclidean is right and Manhattan wrong on 11, the reverse on 3, and on 8 more both are wrong but with different digits. So 58 of the errors are shared and the rest are specific to the distance. Manhattan adds up absolute pixel differences, so many small differences all over the image count as much as a few large ones; Euclidean squares the differences, so it is driven by the pixels that differ a lot, which for digits is the stroke positions. That is why they can rank the training images differently for the same test image. Euclidean wins on this split, but 8 net images out of 500 is not evidence that it always will.

Question 5: Create a plot-formatting skill

5a

I wrote the skill below with my agent (Claude Code) and installed it as

`.claude/skills/format-homework-plots/SKILL.md` in my STAT 432 repo, which is where the agent looks for project skills. The description limits it to STAT 432 homework plots and says it only runs when I ask for it by name.

5b

I asked the agent to apply `format-homework-plots` to the Q1 plot. Original on the left (the figure from 1b as first drawn), revised on the right. Below the figure is what the skill changed.

```
In [11]: def q1_plot_original(ax):
          ax.plot(ks, bias**2, "o-", label="squared bias")
          ax.plot(ks, var, "s-", label="variance")
          ax.plot(ks, mse, "^-", label="MSE")
          ax.set_xlabel("k"); ax.set_ylabel("value"); ax.set_title("Q1: bias-variance of KNN at x0"); ax.legend()

          def q1_plot_revised(ax):
```

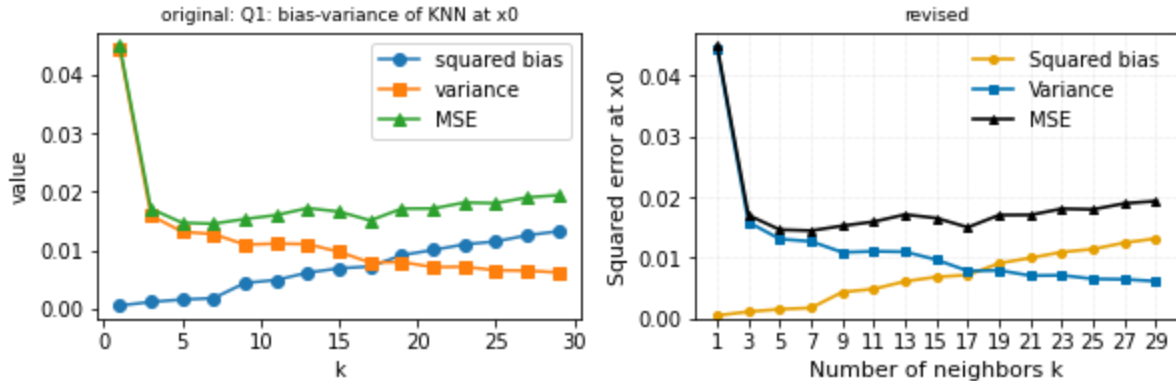
written by the agent under format-homework-plots

```

okabe = {"blue": "#0072B2", "orange": "#E69F00", "black": "#000000"}
ax.plot(ks, bias**2, "o-", color=okabe["orange"], lw=1.6, ms=4, label="Squared bias")
ax.plot(ks, var, "s-", color=okabe["blue"], lw=1.6, ms=4, label="Variance")
ax.plot(ks, mse, "^-", color=okabe["black"], lw=1.6, ms=4, label="MSE")
ax.set_xticks(ks); ax.set_ylim(bottom=0)
ax.set_xlabel("Number of neighbors k", fontsize=11); ax.set_ylabel("Squared error at x0", fontsize=11)
ax.set_title("KNN at x0 = (0.5, 0.7, 1): MSE = squared bias + variance (200 reps)", fontsize=13)
ax.tick_params(labelsize=10); ax.grid(True, ls=":", lw=0.5, alpha=0.6)
ax.legend(frameon=False, loc="upper right", fontsize=10)

fig, axes = plt.subplots(1, 2, figsize=(8.4, 2.9))
q1_plot_original(axes[0]); axes[0].set_title("original: " + axes[0].get_title(), fontsize=9)
q1_plot_revised(axes[1]); axes[1].set_title("revised", fontsize=9)
plt.tight_layout(); plt.show()

```



What changed: the title now says what is plotted (a fixed-target MSE decomposition, 200 repetitions) instead of "Q1"; the axis labels say what k and the y value are; the x ticks sit on the 15 values of k that were actually used instead of matplotlib's 0, 5, 10, ...; the y axis starts at 0; the three series use the Okabe-Ito palette with distinct markers, so they still separate in grayscale; the legend lost its frame; and there is a faint dotted grid. The data are untouched.

Do I like it? Mostly yes. The ticks on the real k grid and the y axis at 0 are the two changes that actually help me read the figure, because the question is "where is the minimum" and "how do the two components trade", and both are easier to see now. The longer title and the axis labels are what I should have written the first time. The grid I am neutral on. The first version of the skill I tested asked for a 6.5×4.5 figure with 14-point titles, which looked fine on screen but wasted space on the 10-page PDF, so I revised the size rule to 6.5×4 for single panels and 3.2×3 for side-by-side ones and dropped the title to 13 pt; the second test produced the figure above and I kept it. I did not uninstall it.

5c

Final `SKILL.md` (installed at `.claude/skills/format-homework-plots/SKILL.md`):

```

---
name: format-homework-plots
description: Improve the formatting of a plot in a STAT 432 homework
notebook (R or Python). Use only when Owen explicitly asks to apply
format-homework-plots to a plot; never on your own, and never for
anything other than STAT 432 homework plots.
---

```

```
# format-homework-plots
```

Goal: make a homework plot easy to read on a printed PDF page without changing what it shows.

Rules

1. Keep the data, the computation and the statistical meaning exactly as they are. Only the presentation changes.
2. Size: a single-panel figure is 6.5 x 4 inches; side-by-side panels are 3.2 x 3 inches each. Never wider than 9 inches, the PDF is letter size with a 10-page limit.
3. Margins: call ``tight_layout()`` (Python) or set ``par(mar = c(4, 4, 2.5, 1))`` (R) so nothing is clipped, and leave no empty white band around the panel.
4. Title: one short line that says what is plotted and against what, not the question number. 13 pt.
5. Axes: label both axes with the quantity and its unit or scale (e.g. "Number of neighbors k", "Test MSE"). 11 pt labels, 10 pt ticks. Ticks at the actual values used when there are few of them (e.g. every k in the grid).
6. Start the y axis at 0 for error and variance quantities unless that hides the pattern; say so in the caption if it does.
7. Colors: at most four series; use a colorblind-safe set (Okabe-Ito: blue #0072B2, orange #E69F00, green #009E73, vermilion #D55E00, black) and give each series a different marker as well as a color, so it still reads in grayscale.
8. Legend inside the plot, in the emptiest corner, no frame.
9. Lines 1.6 pt, markers size 4; no gridlines, or a very light dotted grid only.
10. Return the revised plotting code and a one-line list of what changed.

Why these choices: every rule is something that went wrong in one of my earlier homework plots (clipped labels, "value" as a y label, default colors that vanish when printed), and the rules are about the page the plot ends up on, not about a specific question, so the skill carries over to later weeks. The description is deliberately narrow: I do not want an agent silently restyling figures while I am debugging code, so it has to be called by name. The experience was that the skill is only as good as its numbers; the first sizes I wrote were wrong for a page-limited PDF, and I only found that out by looking at the rendered output, which is the same lesson as with any other code.